

- 行为级仿真工具
- 使用 **task** 任务
- 系统函数和任务
- 预编译命令

- 器件提供商的 **EDA** 工具一般都具有编辑输入，逻辑仿真，逻辑综合，时序仿真等完整功能，包含庞大的器件库，价格昂贵。
- 若仅逻辑仿真，可用轻量级工具，如 **Icarus Verilog**（开源）和 **modelsim**（商用）进行仿真
- 编写 **testbench** 程序来定义测试信号，仿真过程，并输出结果
 - ◆ **testbench** 程序中可用 **\$display** 等系统任务或函数输出结果，读取数据文件，保存输出数据文件等
 - ◆ **testbench** 程序中可用 **`timescale** 等预编译指令定义时间单位精度、包含电路模块文件
 - ◆ **testbench** 程序中可用 **initial**、**always** 语句块定义信号

- Icarus Verilog 编译工具

编译 testbench 程序 `tb.v` 输出二进制代码 `tb.o`

```
iverilog -o tb.o tb.v
```

- vvp 仿真工具

vvp 解释执行二进制 testbench 程序 `tb.o`，并输出程序中定义的波形文件

```
vvp -n tb.o
```

- gtkwave 波形文件查看工具

查看 testbench 定义的 `test.vcd` 波形文件

```
gtkwave test.vcd
```

● Shift74x194.v

```
module shift74x194(                                //Verilog 2001, 2005 syntax
    output reg[3:0] Q,                            // 寄存器输出
    input [1:0] S,                                // 控制输入
    input Ds1, Dsr,                               // 串行输入
    input CP, CR,                                 // 时钟及清零
    input [3:0] D);                               // 并行输入
    always @ (posedge CP, negedge CR) //Verilog 2001, 2005 syntax
        if (~CR) Q <= 4'b0000;
        else
            case (s)
                2'b00: Q <= Q;                    // 保持
                2'b01: Q <= {Q[2:0],Dsr};         // 右移
                2'b10: Q <= {Ds1,Q[3:1]};         // 左移
                2'b11: Q <= D;                    // 并行输入
            endcase
    endmodule
```

● verilog 实现 74x194 电路的仿真测试程序

```
`include "shift74x194.v"
`timescale 1 us / 100 ns
module tb;
    reg [1:0] S;
    reg Ds1, Dsr, CP, CR;
    reg [3:0] D;
    wire [3:0] Q;
    shift74x194 test(Q, S, Ds1, Dsr, CP, CR, D);
    initial
    begin
        S = 2'b00;
        Dsr = 0;
        Ds1 = 0;
        CP = 0;
        CR = 0;
        D = 4'b1100;
        $display("Running testbench");
        #5 CR = 1;    //5 个单位时间后
        #1000 $finish; //1000 个单位时间后结束仿真
    end
```

```
initial $monitor("test OK");
initial
begin
    $dumpfile("tb.vcd");
    $dumpvars(0, tb);
end
always #1 CP = ~CP;
always #4 Dsr = ~Dsr;
always #6 Ds1 = ~Ds1;

always
begin
    #20 S[0] = ~S[0];
    #20 S[1] = ~S[1];
end
endmodule
```

- 使用 **task** 封装重复操作，提高代码利用率

```
reg [7:0] ram_mem [10'h3ff:0];
reg [7:0] rom_mem [13'h1fff:0];
initial
begin
    sys_reset;
    dat_load;
    $finish;
end

task sys_reset;
begin
    reset_req = 0;
    #10 reset_req = 1;
    #30 reset_req = 0;
end
endtask

task dat_load;
begin
    $readmemb("test.pro", rom_mem);
    $display("rom loaded successfully");
    $readmemb("test.dat", ram_mem);
    $display("ram loaded successfully");
    #10000;
    sys_reset;
end
endtask
```

- Verilog 预先定义的函数或任务，用于调试和编译预处理。
- 以 \$ 开头，用于控制和检测仿真模拟过程
- 系统函数和任务主要包括：
 - （1）获取仿真时间的系统函数
 - （2）文本输出（检测信号、显示信号）的系统任务
 - （3）文件输入、输出操作的系统任务
 - （4）暂停和退出仿真的系统任务
 - （5）产生随机数的系统任务



1.3.1 获取当前仿真时间与随机数的系统函数

● 获取当前时间

- ◆ **\$time** 返回 64 位的整数时间值；
- ◆ **\$realtime** 返回的结果是实数时间值，是更精确的仿真时间
- ◆ **\$stime** 返回 32 位整数时间值。
- ◆ 函数返回值使用 ``timescale` 定义的时间单位

● 设置时间输出格式

- ◆ **\$timeformat**(单位，精度，文本，字符数)
- ◆ **\$timeformat**(-9, 1, “ns”, 12);

● 产生随机数

- ◆ **Number = \$random**; // 返回一个 32 位的有符号随机数
- ◆ **Number = \$random(seed)**; // 根据 seed，返回随机数



- 输出文本信息

- ◆ **\$display<b/o/h>** : 输出格式化文本, 自动换行
- ◆ **\$write<b/o/h>** : 输出格式化文本, 不自动换行
- ◆ **\$strobe<b/o/h>** : 探测时间拍内指定信号的变化, 并在拍末输出格式化文本
- ◆ **\$monitor** : 持续监视指定信号的变化, 并在拍末输出文本
- ◆ **\$monitoron, \$monitoroff**: 启动, 关闭监控

1.3.2 文本输出的系统任务

- `$display($time, "%b \t %h \t %d \t %o", s1, s2, s3, s4);`
- `$display($time, "%b \t", s1, "%h \t", s2, "%o", s3);`
- `$write($time, "%b \t %h \t %d \t %o\t/n", s1, s2, s3, s4);`
- `$monitor($time, "a=%d, b=%d", a, b);`

格式符:

<code>%h</code>	<code>%o</code>	<code>%d</code>	<code>%b</code>	<code>%c</code>	<code>%s</code>	<code>%v</code>	<code>%m</code>	<code>%t</code>
hex	octal	decimal	binary	ASCII	string	strength	module	time

其中 `%v` 输出三个字符

前两位: 强度 (Su/St/Pu/We/Hi, 即供电 / 强驱 / 上下拉级 / 弱驱 / 高阻级)

最后一位: 逻辑值 (0/1/X/Z)

转义符:

<code>\t</code>	<code>\n</code>	<code>\\</code>	<code>\"</code>	<code>%%</code>	<code>%0d</code>
tab	换行	反斜杠	双引号	百分符号 %	无前导 0 的十进制数



- 文件的打开和关闭:

- ◆ `$fopen`, `$fclose`

- ◆ `Descriptor = $fopen("filename");`

- `$fclose(descriptor);`

- ◆ 返回 32 位无符号文件描述符, 若打开失败值为 0

- 文件格式化文本输出任务:

- ◆ `$fdisplay/$fwrite/$fmonitor/$fstrobe`

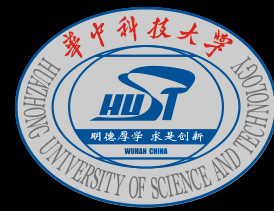
- **\$readmemb**, **\$readmemh**
- 将文件中的二进制或十六进制数读到存储器（即寄存器数组中）
- 语法表达方式：

\$readmemb("filename",memname<,start address><,finish Address>;

\$readmemh("filename",memname<,start address><,finish Address>;

- 文件格式：必须是二进制或十六进制数，数与数之间用空白符隔开，一个数之间可用下画线

1.3.5 暂停和退出仿真的系统任务



- **\$stop** 暂停仿真过程

- ◆ 运行到 **\$stop** 时，暂停仿真；此时可在命令行输入 **run** 继续运行仿真

- **\$finish** 退出仿真过程

- **\$dumpfile**("file_name")

- ◆ 创建记录各种信号变化的信息记录文件，即波形记录文件（VCD, Value Change Dump, IEEE 标准数据库文件）

- **\$dumpvars**(var_level, module_name)

- ◆ 指定需要记录到 VCD 文件中的信号
- ◆ var_level: 记录指定模块之下 var_level 层的信号，0 表示指定模块之下所有层的信号
- ◆ module_name: 指定的模块
- ◆ \$dumpvar (0, top); 记录 top 模块及以下所有层的信号
- ◆ \$dumpvar (2, top); 记录 top 模块及下一层的信号

- 预编译指令都以 ``` 开头
- ``include` “file_name”: 包含另外的源文件
- ``timescale` 仿真时间单位 / 时间精度
- ``define` 和 ``undef`: 宏定义和宏取消
- ``ifdef`、``else`、``endif`: 条件编译
 - ◆ 根据宏条件, 选择代码段进行编译

- 示例:

```
`define wordsize 8  
reg [1:`wordsize] data;
```

```
wire a, b, c, d, out;  
`define exp a & b & c;  
assign out = `exp & d;
```

```
`define WOW  
`ifdef WOW  
    initial $display("wow is defined");  
`else  
    initial $display("wow is not defined");  
`endif
```


1.5 时序控制



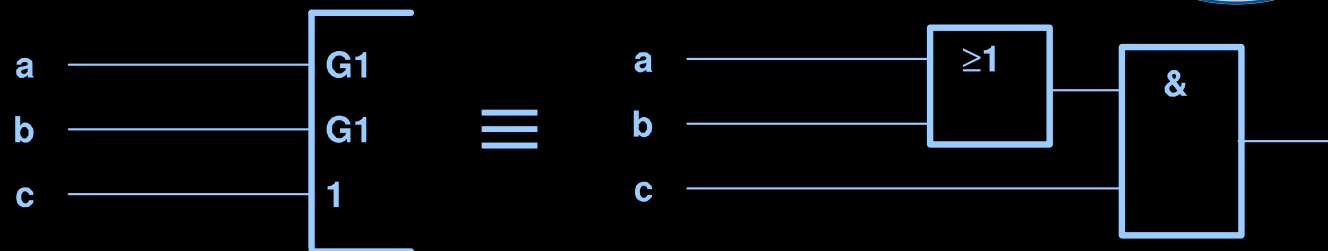
```
`timescale 1us/100ns
module begin_end;
reg a, b, clk;
initial
begin
    a = 0;
    b = 0;
    clk = 0;
    #1 clk = 1;
end
always @(posedge clk)
begin
    #2 a = 1;
    #1 b = a;
end
initial $monitor("test OK");
initial
begin
    $dumpfile("tb.vcd");
    $dumpvars(0, tb);
end
endmodule
```



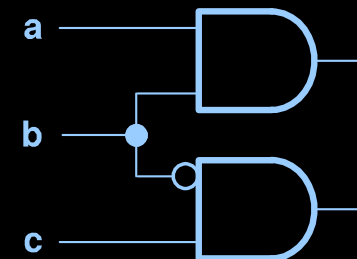
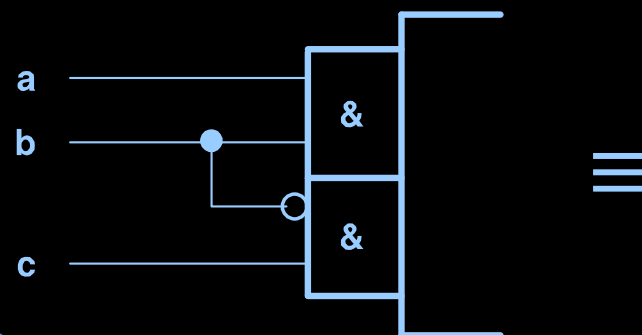
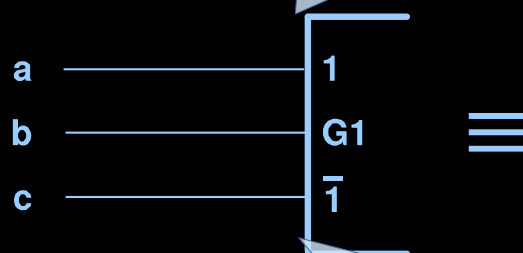
- 数字编号端 n 或 $\overline{n}$ 的信号依赖于
 - ◆ C_n 端信号 (或反相信号) 控制
 - ◆ X_n 端信号 (或反相信号) 传输
 - ◆ Z_n 端信号 (或反相信号) 时延
 - ◆ M_n 端信号 (或反相信号) 模式
 - ◆ A_n 端信号 (或反相信号) 地址
 - ◆ S_n 端信号 (或反相信号) 置 1
 - ◆ R_n 端信号 (或反相信号) 置 0
 - ◆ EN_n 端信号 (或反相信号) 使能
 - ◆ G_n 端信号 (或反相信号) 相与
 - ◆ V_n 端信号 (或反相信号) 相或
 - ◆ N_n 端信号 (或反相信号) 异或
- 多重依赖用 , 号分隔
- 可选依赖用 / 号分隔

2 ANSI/IEEE 通用逻辑依赖标记

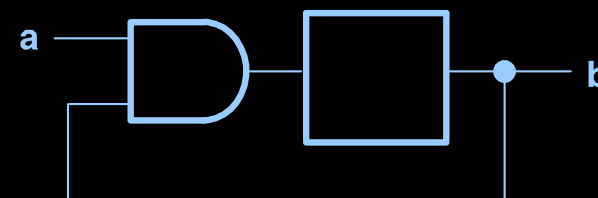
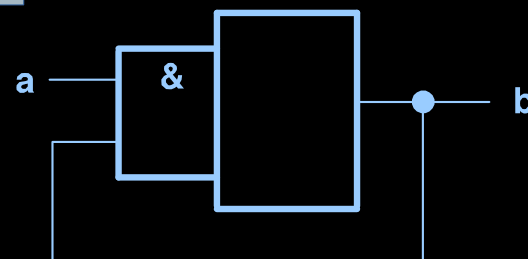
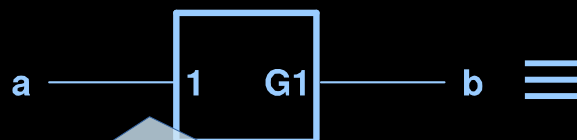
1 端信号依赖于 G1 端信号之或相与



1 端信号依赖于 G1 端信号相与

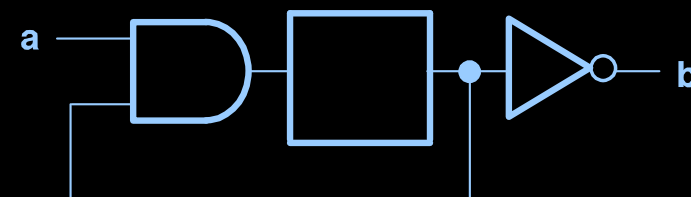
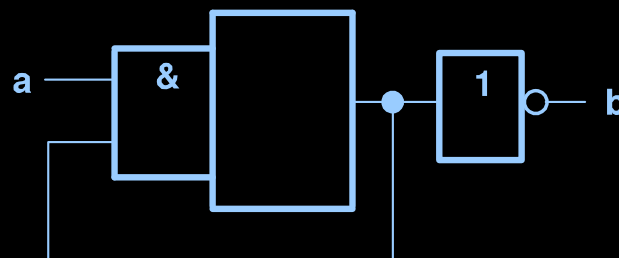


1 端信号依赖于 G1 端反相信号相与



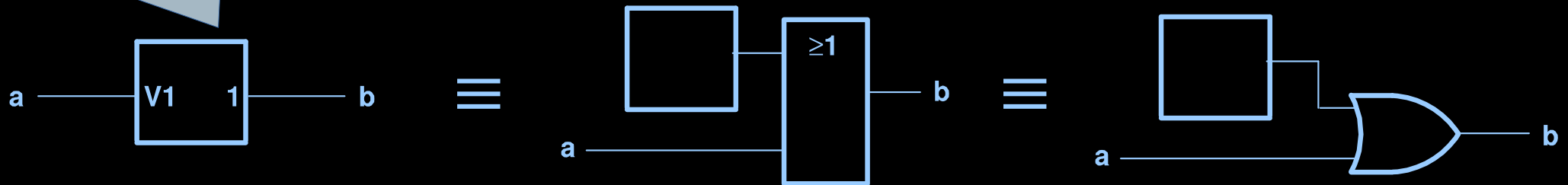
1 端信号依赖于 G1 端信号相与

1 端信号依赖于 G1 端信号相与
b 是 G1 端反相信号

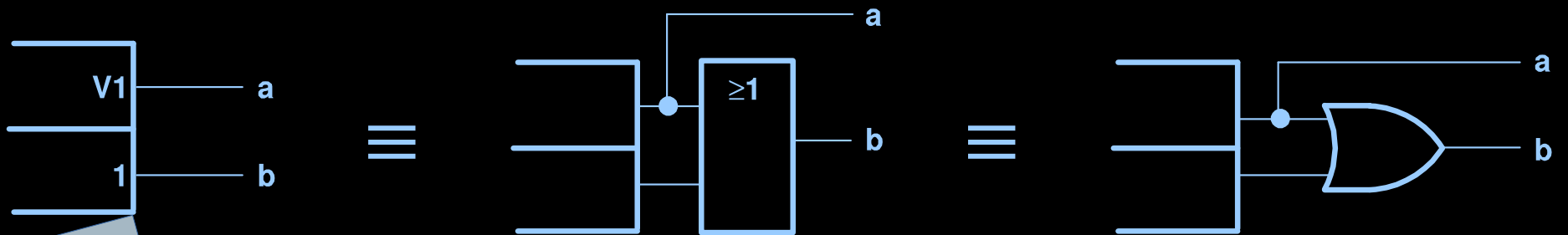


2 ANSI/IEEE 通用逻辑依赖标记

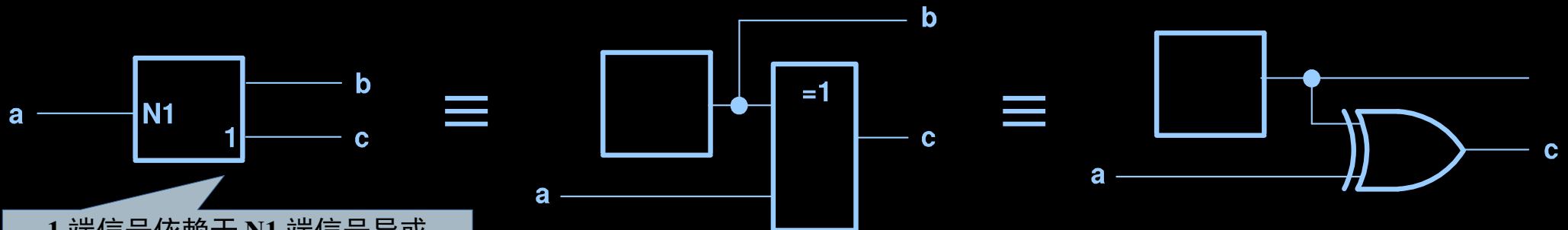
1 端信号依赖于 V1 端信号相或



1 端信号依赖于 V1 端信号相或

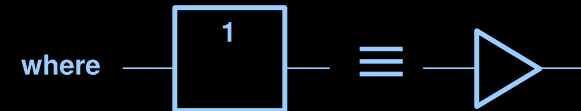
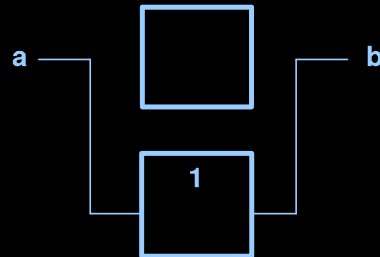
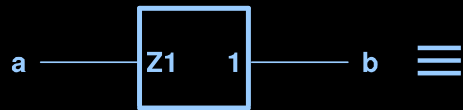


1 端信号依赖于 N1 端信号异或

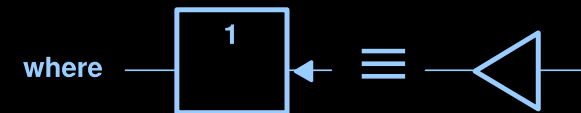
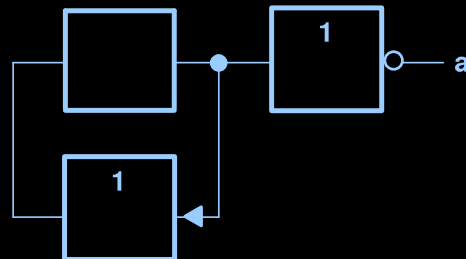
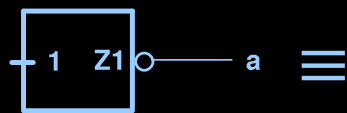


2 ANSI/IEEE 通用逻辑依赖标记

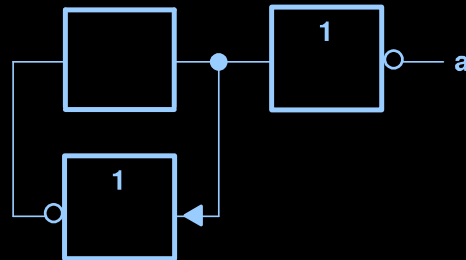
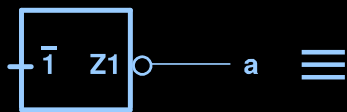
1 端信号依赖于 Z1 端信号时延



1 端信号依赖于 Z1 端信号时延
 a 是 Z1 端反相信号



$\bar{1}$ 端信号依赖于 Z1 端反相信号时延
 a 是 Z1 端反相信号



2 端信号依赖于 1Z2 端信号时延
1Z2 端信号依赖于 G1 端信号相与

